

MVC PHP TD2

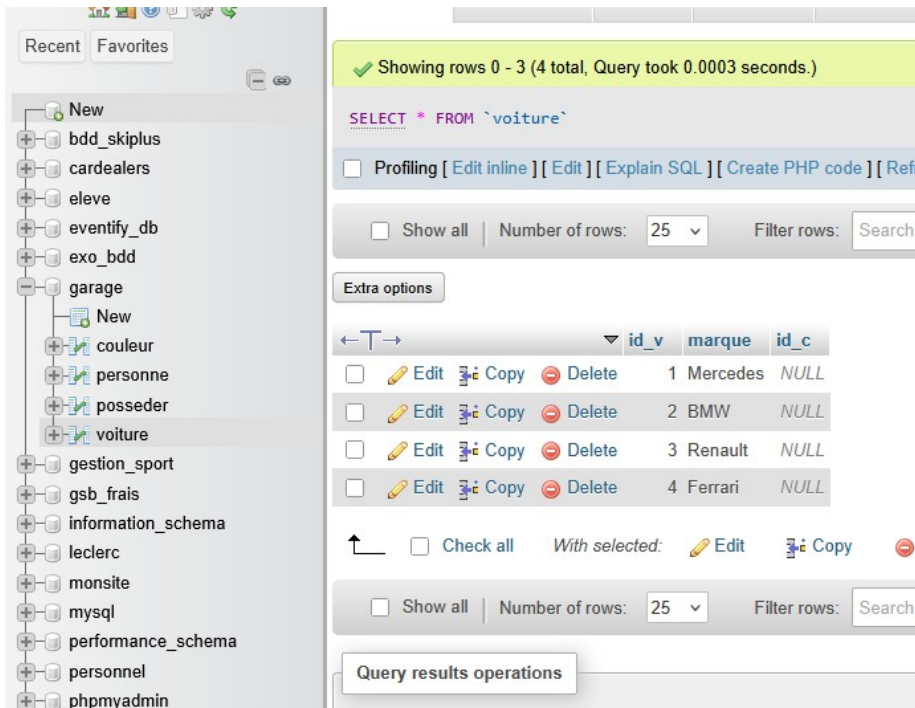
Sommaire

<u>Introduction</u>	1
<u>Exercice:1</u>	2
<u>Exercice :2</u>	2
<u>Exercice:3</u>	4
<u>Exercice :4</u>	5
<u>Exercice:5</u>	6
<u>Exercice : 6</u>	6
<u>Exercice:7</u>	7
<u>Exercice : 8</u>	8
<u>Exercice : 9</u>	8
<u>Conclusion</u>	8

Introduction

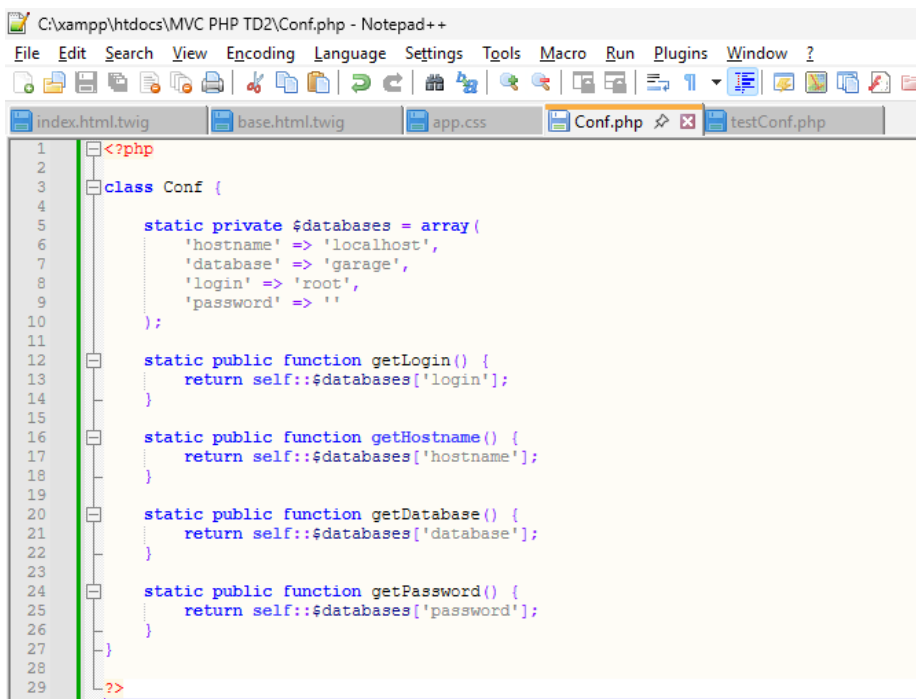
Dans ce TP, nous allons revoir les bases de PHP et de la programmation orientée objet. Nous allons apprendre à connecter PHP à une base de données MySQL avec PDO afin de sauvegarder et récupérer des objets. Nous appliquerons ensuite ces notions à un exemple de site de covoiturage.

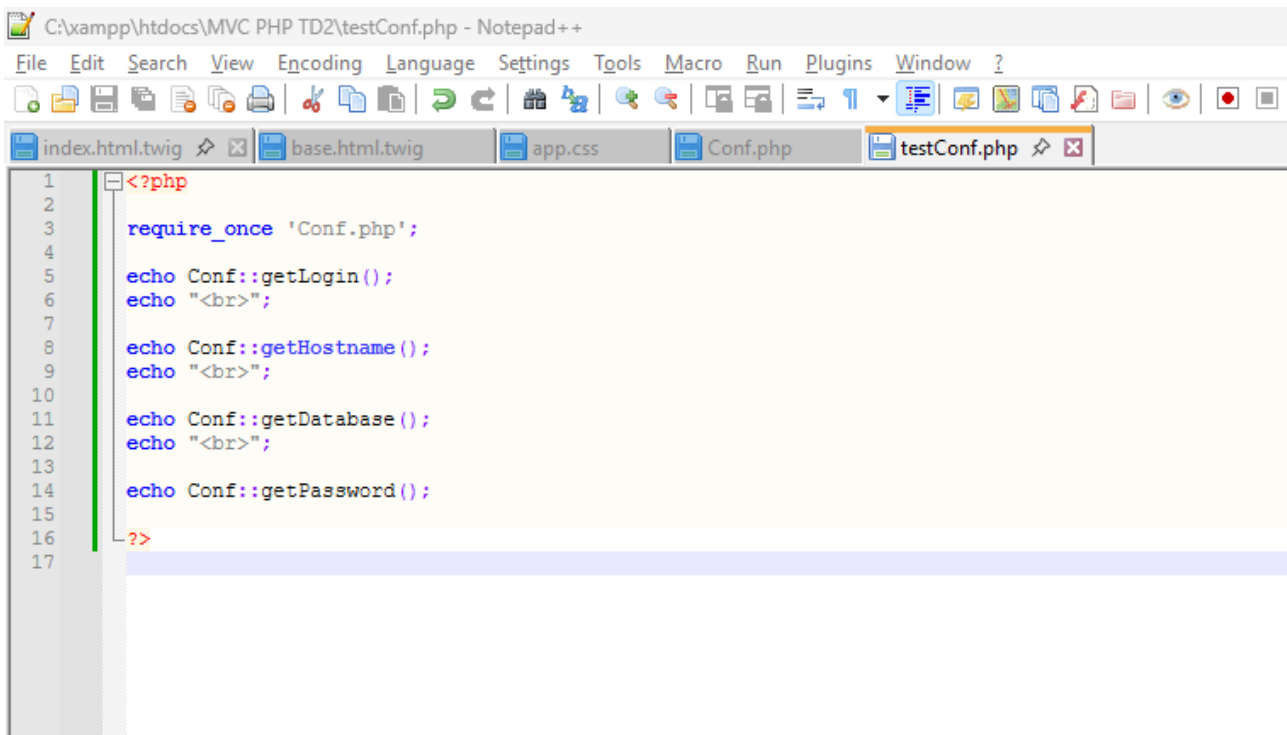
Exercice:1



Exercice :2

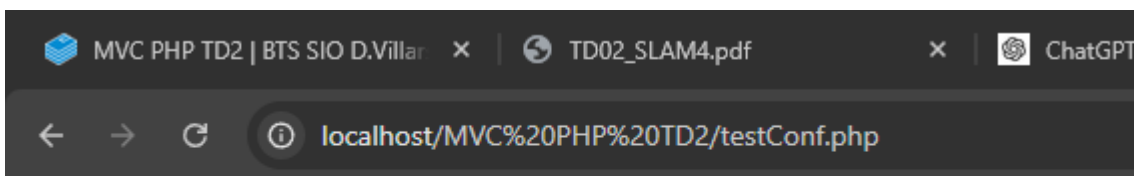
J'ai créé les deux fichiers Conf.php et testConf.php pour configurer et tester la connexion à ma base de données. Voici les fichiers ainsi que le résultat obtenu sur localhost.





The screenshot shows the Notepad++ application window titled "C:\xampp\htdocs\MVC PHP TD2\testConf.php - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and ?. The toolbar contains various icons for file operations and editing. The tab bar shows four open files: index.html.twig, base.html.twig, app.css, and testConf.php. The main editing area displays the following PHP code:

```
1 <?php
2
3 require_once 'Conf.php';
4
5 echo Conf::getLogin();
6 echo "<br>";
7
8 echo Conf::getHostname();
9 echo "<br>";
10
11 echo Conf::getDatabase();
12 echo "<br>";
13
14 echo Conf::getPassword();
15
16 ?>
17
```



Exercice:3

J'ai initialisé Git dans mon projet et enregistré mon travail avec un commit. Le git status confirme ensuite que tout est bien sauvegardé.

```
C:\WINDOWS\system32\cmd. X + v
C:\xampp\htdocs\MVC PHP TD2>git status
fatal: not a git repository (or any of the parent directories): .git

C:\xampp\htdocs\MVC PHP TD2>git init
Initialized empty Git repository in C:/xampp/htdocs/MVC PHP TD2/.git/

C:\xampp\htdocs\MVC PHP TD2>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Conf.php
    testConf.php

nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\MVC PHP TD2>git add .

C:\xampp\htdocs\MVC PHP TD2>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   Conf.php
    new file:   testConf.php

C:\xampp\htdocs\MVC PHP TD2>
```

```
C:\xampp\htdocs\MVC PHP TD2>git commit -m "Exercices 1 et 2"
[master (root-commit) a0a7399] Exercices 1 et 2
 2 files changed, 45 insertions(+)
 create mode 100644 Conf.php
 create mode 100644 testConf.php

C:\xampp\htdocs\MVC PHP TD2>git status
On branch master
nothing to commit, working tree clean

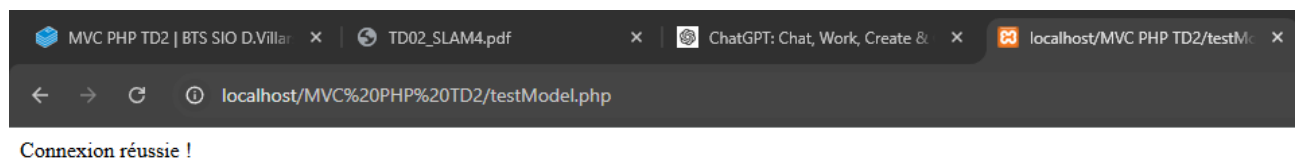
C:\xampp\htdocs\MVC PHP TD2>
```

Exercice :4

J'ai créé la classe Model pour établir la connexion entre PHP et ma base de données grâce à PDO. J'ai ensuite testé la connexion avec testModel.php et celle-ci fonctionne correctement.

```
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
index.html.twig base.html.twig app.css Conf.php testConf.php Model.php testModel.php
1 <?php
2
3 require_once 'Conf.php';
4
5 class Model {
6
7     public static $pdo;
8
9     public static function Init() {
10         $hostname = Conf::getHostname();
11         $database_name = Conf::getDatabase();
12         $login = Conf::getLogin();
13         $password = Conf::getPassword();
14
15         self::$pdo = new PDO(
16             "mysql:host=$hostname;dbname=$database_name",
17             $login,
18             $password
19         );
20     }
21 }
22
23 Model::Init();
24
25 ??
26
```

```
index.html.twig base.html.twig app.css Conf.php testConf.php Model.php testModel.php
1 <?php
2
3 require_once 'Model.php';
4
5 echo "Connexion réussie !";
6
7 ??
8
```

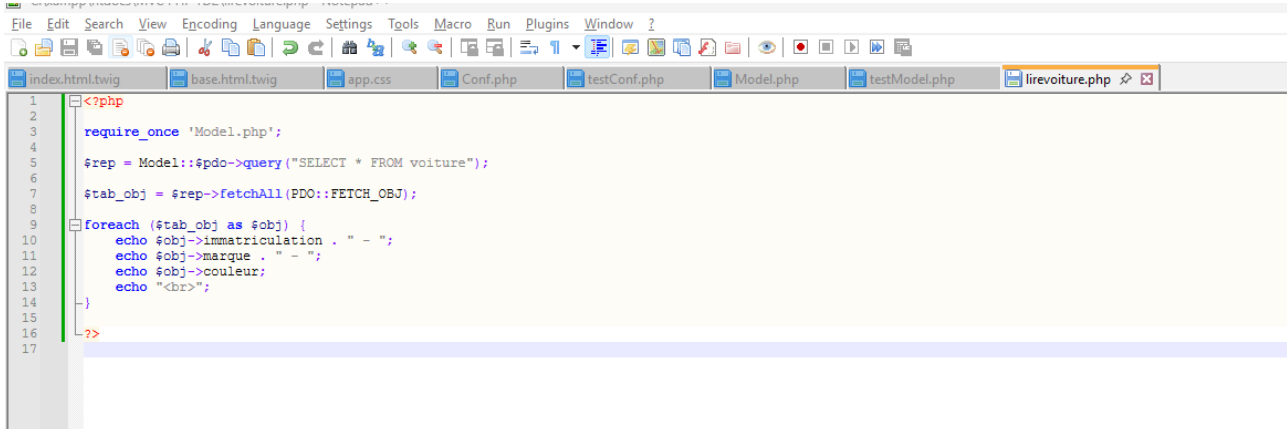


Exercice:5

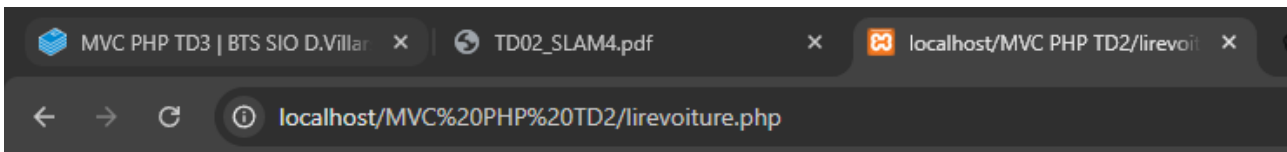
Dans cet exercice, j'ai amélioré la connexion PDO en ajoutant la gestion des erreurs et la prise en charge de l'UTF-8. Le résultat reste le même lorsque la connexion fonctionne, mais le code est maintenant mieux géré en cas de problème.

Exercice : 6

J'ai créé une requête SQL avec PDO pour récupérer toutes les voitures de la base de données. Les résultats sont ensuite affichés avec une boucle foreach.



```
1 <?php
2
3 require_once 'Model.php';
4
5 $rep = Model::$pdo->query("SELECT * FROM voiture");
6
7 $tab_obj = $rep->fetchAll(PDO::FETCH_OBJ);
8
9 foreach ($tab_obj as $obj) {
10     echo $obj->immatriculation . " - ";
11     echo $obj->marque . " - ";
12     echo $obj->couleur;
13     echo "<br>";
14 }
15
16
17
```



AB123CD - Mercedes - Noir
EF456GH - BMW - Blanc
IJ789KL - Renault - Rouge
MN123OP - Ferrari - Jaune
QR456ST - Audi - Gris
UV789WX - Peugeot - Bleu

Exercice:7

J'ai utilisé la classe Voiture avec PDO pour récupérer les voitures sous forme d'objets. J'ai ensuite utilisé la méthode afficher() pour afficher leurs informations.

```

c:\xampp\htdocs\MVC PHP TD2\voiture.php - Notepad++
Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
ex.html.twig base.html.twig app.css Conf.php testConf.php Model.php testModel.php lirevoiture.php voiture.php
class Voiture {
    public $immatriculation;
    public $marque;
    public $couleur;

    public function afficher() {
        echo $this->immatriculation . " - ";
        echo $this->marque . " - ";
        echo $this->couleur;
    }
}

```

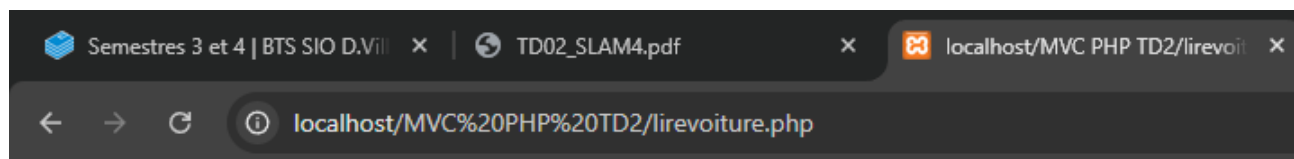
```

Search View Encoding Language Settings Tools Macro Run Plugins Window ?
nl.twig base.html.twig app.css Conf.php testConf.php Model.php testModel.php lirevoiture.php
<?php
require_once 'Model.php';
require_once 'voiture.php';

$rep = Model::$pdo->query("SELECT * FROM voiture");
$rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
$tab_voit = $rep->fetchAll();

foreach ($tab_voit as $voit) {
    $voit->afficher();
    echo "<br>";
}

```



AB123CD - Mercedes - Noir
EF456GH - BMW - Blanc
IJ789KL - Renault - Rouge
MN123OP - Ferrari - Jaune
QR456ST - Audi - Gris
UV789WX - Peugeot - Bleu

Exercice : 8

Le résultat reste le même, mais le code est mieux organisé. La requête SQL a été déplacée dans la classe Voiture grâce à la méthode getAllVoitures().

Exercice : 9

Conclusion

Ce TP nous a permis de revoir PHP, les classes et les objets, tout en apprenant à utiliser une base de données avec PDO. Nous avons également appris à récupérer des données et à les utiliser dans notre application de covoiturage.